

CSE 406
Design Report

**TCP Session Hijacking via Command
Injection**

Student 1: Sadia Tasnim

ID: 2105077

Student 2: Mahbuba Sayed Sinthia

ID: 2105078

1. Overview of the Project Idea

TCP session hijacking is an attack in which an attacker who can observe an established TCP connection injects a forged TCP segment that impersonates one of the two endpoints. Because TCP identifies a connection only by the 4-tuple (source IP, source port, destination IP, destination port) and accepts data whose sequence number falls within the receiver's current window, a segment that carries the right tuple and the right sequence number is treated as genuine — TCP performs no per-segment authentication of the sender.

In this project we build a small custom TCP application: a server that accepts plain-text commands and a client (the victim) that connects and issues legitimate commands. A third machine, the attacker, sits on the same network segment, sniffs the live session, and injects a forged segment carrying a command of its choosing (for example an "admin" command the client would never send). The server processes the injected command as if it came from the client. We use a custom plaintext application protocol rather than a real HTTP server because modern HTTP stacks carry protections that obscure the core mechanism; a purpose-built protocol lets the sequence-injection behaviour show clearly.

2. Network Topology and Components

Three virtual machines run on one host under VirtualBox using Internal Network mode (isolated, no NAT/bridge to the outside), with each adapter's Promiscuous Mode set to "Allow All." The internal network behaves like a shared switch; with promiscuous mode permitted, the attacker's NIC can capture frames exchanged between client and server. This is what places the attacker "on-path" without any extra step.

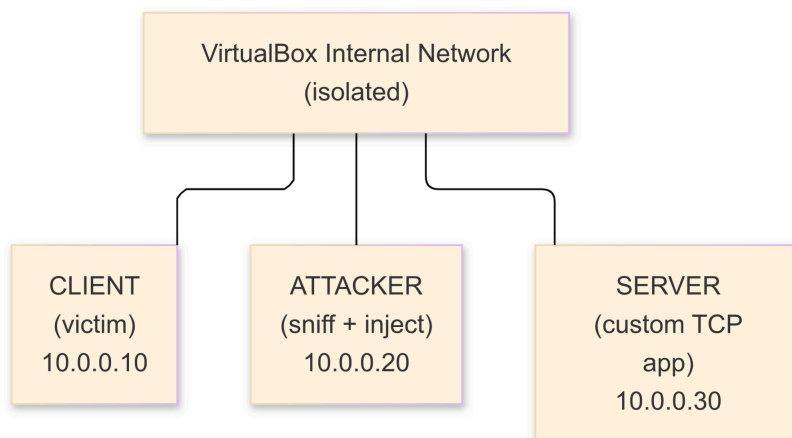
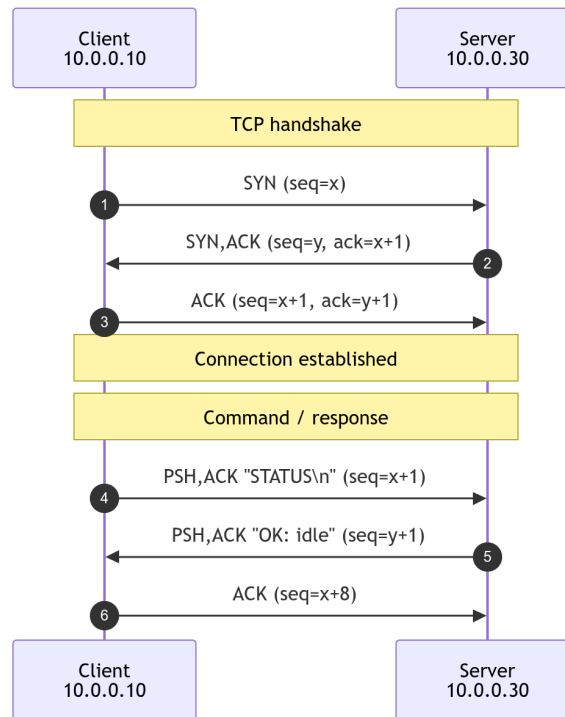


Fig: Network Topology

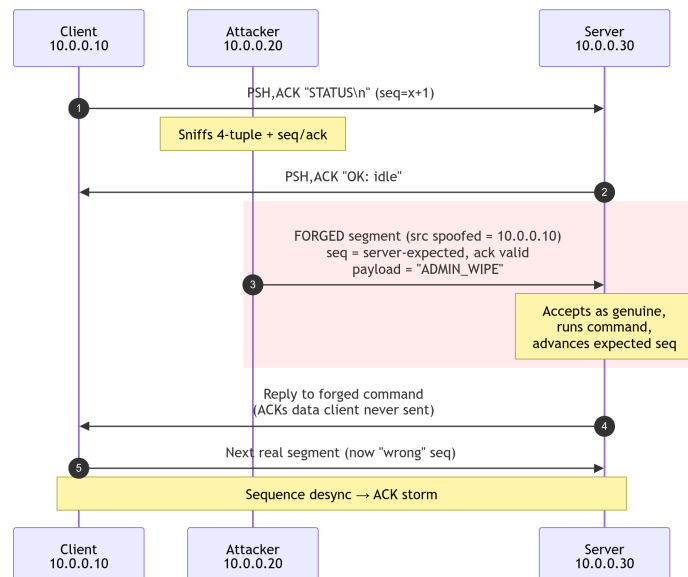
Components: Client 10.0.0.10 (issues legitimate commands), Server 10.0.0.30:8080 (accepts and replies to commands), Attacker 10.0.0.20 (passively sniffs, then actively injects). All IPs are statically assigned.

3. Timing Diagrams

Original protocol (no attacker). Normal handshake, then a command/response exchange.



Attack. The attacker sniffs the exchange, learns the 4-tuple and the server's currently-expected sequence number, then races a forged segment ahead of the client's next legitimate one.



Attack strategy. The forged segment reuses the client's exact source IP and port and carries the sequence number the server is waiting for. Once the server accepts it and advances its expected sequence value, the real client's subsequent segment no longer matches, producing sequence desynchronization and a burst of corrective ACKs (the "ACK storm"), which is itself a useful piece of demo evidence.

4. Packet and Frame Structure

The attacker constructs an IP + TCP segment whose header fields match the sniffed session, changing only what's needed to impersonate the client and deliver the payload.

Field	Legitimate client segment	Forged (attacker) segment
Source IP	10.0.0.10	10.0.0.10 (spoofed = client)
Destination IP	10.0.0.30	10.0.0.30
Source Port	49152 (client ephemeral)	49152 (copied from sniff)
Destination Port	8080	8080
Sequence number	client's current seq	set to server's next-expected seq
Ack number	server's next-expected	valid value taken from sniff
Flags	PSH, ACK	PSH, ACK
Window	client's window	plausible value copied from session
Payload	e.g. STATUS\n	e.g. ADMIN_WIPE\n (attacker command)
IP/TCP checksum	valid	recomputed after edits

Header modifications that matter: the spoofed source IP, the reused 4-tuple (must match exactly or the server drops it), and above all the sequence number aligned to what the server currently expects. Because the payload and headers change, both the IP and TCP checksums are recomputed. Note that the TCP checksum is computed over a pseudo-header (source IP, destination IP, protocol, TCP length) prepended to the segment, not over the TCP header alone — this is a detail our implementation handles explicitly.

5. Why the Design Should Work (Justification)

Three properties combine to make injection succeed. First, TCP authenticates nothing at the segment level — a segment is accepted purely on a matching 4-tuple and an in-window sequence number. Second, the attacker is on-path (shared internal network + promiscuous mode), so it reads the live sequence/ack numbers directly instead of guessing; this removes the hardest part of a classic hijack, blind ISN prediction, and lets the forged segment carry the precise sequence value the server expects. Third, the custom application protocol is plaintext with no integrity or authentication at the application layer, so an injected command is indistinguishable from a genuine one.

The one assumption worth stating: the attack depends on the attacker being on-path. The VirtualBox Internal Network with promiscuous mode provides this directly; on a real switched LAN the attacker would first need to become on-path (e.g. via ARP spoofing) before this technique applies.

6. Implementation Plan

The server and client are small custom programs using Python's socket library: the server listens on 10.0.0.30:8080, reads text commands, and replies; the client connects and sends commands in a loop so there is live traffic to hijack.

The attacker tool will be written from scratch using Python's standard library — `socket` for raw socket access and `struct` for byte packing. Every IP and TCP header is expected to be packed field-by-field in our own code and both checksums are computed by our own functions, satisfying the requirement to craft our own packets. The tool works in two stages:

1. **Sniff.** An `AF_PACKET` raw socket captures the full Ethernet frames the attacker's NIC sees (this is why promiscuous mode is required). Our code parses Ethernet → IP → TCP by hand to extract the session's 4-tuple, sequence number, ack number, and window.
2. **Inject.** From a sniffed client→server data segment carrying sequence `S` and `L` payload bytes, the tool computes the server's next-expected sequence as `S + L`. It then hand-builds an IP header and a TCP header (spoofing the client's source IP, reusing the 4-tuple, setting `seq = S + L`), recomputes both checksums, and sends the forged segment through an `AF_INET` raw socket opened with `IPPROTO_RAW` (so the kernel uses our IP header rather than generating its own).

Wireshark runs on the server or attacker throughout to capture evidence. The tool requires root privileges on the attacker VM (raw sockets need `CAP_NET_RAW`) and targets Linux (`AF_PACKET` is Linux-specific).

7. Expected Outcome of a Successful Attack

On success, the server accepts the forged segment and executes the attacker's command as though it came from the client: the server log shows the injected command arriving from the client's IP, and the server replies to it. The client and server sequence numbers desynchronize, visible in Wireshark as duplicate ACKs / an ACK storm, and the victim client's session hangs or breaks. Evidence for the final report: the attacker terminal showing the crafted segment being sent; the server log showing the unexpected command; a Wireshark capture showing the forged segment (client's IP, but timed with the attacker's injection); and the client terminal showing session disruption.